Laboratory 1: Project Setup and REST Basics

Desktop Application Programming II - Backend Development

Duration: 90 minutes
Topics Covered: Lectures 1-2 (Introduction to Backend Systems, RESTful APIs, HTTP Fundamentals)

Learning Objectives

By the end of this lab, you will:

- Set up a Spring Boot project with proper structure
- Understand basic REST endpoint creation
- Implement simple controllers with different HTTP methods
- Test API endpoints using Postman or similar tools
- Choose and get approval for your semester project topic

Task 1: Project Topic Selection

Description

Choose your project topic from the suggested list or propose your own idea. Your project will be developed throughout the semester.

Requirements

- Topic must be different from your Android/Flutter projects
- Must have realistic scope (achievable in 6 weeks)
- Should interest you personally
- Must include at least 3 main entities

Action Items

1. Review the project requirements document
2. Think about a domain you'd like to work with
3. Discuss your choice with the instructor for approval
4. Document your decision:
    - Project name
    - Brief description (2-3 sentences)
    - Main entities you plan to implement

Task 2: Spring Boot Project Initialization

Description

Create a new Spring Boot project with all necessary dependencies.

Option A: Using Spring Initializr (Recommended)

1. Go to https://start.spring.io/
2. Configure your project:
    - **Project:** Maven
    - **Language:** Java

- **Spring Boot:** 3.2.x (latest stable)
- **Group:** com.yourname (e.g., com.johndoe)
- **Artifact:** your-project-name (e.g., task-manager)
- **Packaging:** Jar
- **Java:** 17 or 21

3. Add dependencies:
   - Spring Web
   - Spring Data JPA
   - Spring Security (you can disable it for now)
   - PostgreSQL Driver (or MySQL)
   - H2 Database (for development)
   - Lombok
   - Validation
4. Click "Generate" and download the ZIP file
5. Extract and open in IntelliJ IDEA

Option B: Using IntelliJ IDEA

1. File → New → Project
2. Select "Spring Initializr"
3. Configure as described above
4. Click "Next" and add dependencies
5. Finish and create project

Verification

- Project opens without errors
- Maven dependencies download successfully
- You can see the main application class with `@SpringBootApplication`

Task 3: Basic Project Configuration

Description

Configure your application properties and verify Spring Boot can start.

Action Items

1. Open `src/main/resources/application.properties`
2. Add basic configuration:

```
# Application name
spring.application.name=your-project-name

# Server port
server.port=8080

# H2 Database (for now)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# H2 Console
```

```
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA/Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# Disable security for now (remove this later)
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfigu
```

3. Run your application:
   - Find the main class (ends with `Application.java`)
   - Right-click → Run
   - Check console for successful startup
4. Verify startup:
   - Look for: "Started YourApplication in X seconds"
   - No error messages in console
   - Application running on port 8080

Task 4: Create Your First REST Controller

Description

Create a simple controller with basic endpoints to understand REST fundamentals.

Action Items

1. Create package structure:

```
src/main/java/com/yourname/projectname/
   controller/
   service/
   repository/
   model/
   dto/
```

2. Create a test controller: `controller/HelloController.java`

```java
package com.yourname.projectname.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/hello")
public class HelloController {

    @GetMapping
    public ResponseEntity<String> sayHello() {
        return ResponseEntity.ok("Hello from Spring Boot!");
    }

    @GetMapping("/{name}")
    public ResponseEntity<String> sayHelloToName(@PathVariable String name) {
```

```
        return ResponseEntity.ok("Hello, " + name + "!");
    }

    @PostMapping
    public ResponseEntity<String> receiveMessage(@RequestBody String message) {
        return ResponseEntity.ok("You sent: " + message);
    }
}
```

3. Test your endpoints:
   - Start your application
   - Open Postman (or use curl)
   - Test: GET `http://localhost:8080/api/hello`
   - Test: GET `http://localhost:8080/api/hello/John`
   - Test: POST `http://localhost:8080/api/hello` with body: `"Test message"`

Task 5: Create Domain-Specific Controller

Description

Create a controller for one of your main entities with basic CRUD structure (no database yet).

Example: For a Task Management System

1. Create a simple model class: `model/Task.java`

```
package com.yourname.projectname.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Task {
    private Long id;
    private String title;
    private String description;
    private boolean completed;
}
```

2. Create controller: `controller/TaskController.java`

```
package com.yourname.projectname.controller;

import com.yourname.projectname.model.Task;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;
```

```java
@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    // Temporary in-memory storage
    private final List<Task> tasks = new ArrayList<>();
    private final AtomicLong idCounter = new AtomicLong(1);

    @GetMapping
    public ResponseEntity<List<Task>> getAllTasks() {
        return ResponseEntity.ok(tasks);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Task> getTaskById(@PathVariable Long id) {
        return tasks.stream()
                .filter(task -> task.getId().equals(id))
                .findFirst()
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Task> createTask(@RequestBody Task task) {
        task.setId(idCounter.getAndIncrement());
        tasks.add(task);
        return ResponseEntity.status(HttpStatus.CREATED).body(task);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Task> updateTask(
            @PathVariable Long id,
            @RequestBody Task updatedTask) {
        for (int i = 0; i < tasks.size(); i++) {
            if (tasks.get(i).getId().equals(id)) {
                updatedTask.setId(id);
                tasks.set(i, updatedTask);
                return ResponseEntity.ok(updatedTask);
            }
        }
        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteTask(@PathVariable Long id) {
        boolean removed = tasks.removeIf(task -> task.getId().equals(id));
        return removed ?
            ResponseEntity.noContent().build() :
            ResponseEntity.notFound().build();
    }
}
```

3. Adapt this example to **your chosen entity**:
    - Change class names

- Adjust fields
- Keep the CRUD structure

Task 6: Test Your API with Postman

Description

Create a Postman collection to test all your endpoints.

Action Items

1. Download and install Postman (if not already installed)
2. Create a new collection: "Your Project Name - Lab 1"
3. Add requests for each endpoint:

   GET All Items

   - Method: GET
   - URL: `http://localhost:8080/api/tasks`

   GET Single Item

   - Method: GET
   - URL: `http://localhost:8080/api/tasks/1`

   POST Create Item

   - Method: POST
   - URL: `http://localhost:8080/api/tasks`
   - Body (JSON):

```
{
    "title": "Complete Lab 1",
    "description": "Finish all tasks for laboratory 1",
    "completed": false
}
```

   PUT Update Item

   - Method: PUT
   - URL: `http://localhost:8080/api/tasks/1`
   - Body (JSON):

```
{
    "title": "Complete Lab 1",
    "description": "Finish all tasks for laboratory 1",
    "completed": true
}
```

   DELETE Item

   - Method: DELETE
   - URL: `http://localhost:8080/api/tasks/1`

4. Test the complete flow:

   - Create several items
   - Get all items
   - Get single item by ID

- Update an item
- Delete an item
- Verify 404 for non-existent items

Task 7: Initialize Git Repository

Description

Set up version control for your project.

Action Items

1. Initialize Git in your project directory:

   ```
   git init
   ```

2. Create `.gitignore` file (if not exists):

   ```
   # Maven
   target/
   pom.xml.tag
   pom.xml.releaseBackup
   pom.xml.versionsBackup

   # IntelliJ IDEA
   .idea/
   *.iml
   *.iws

   # Eclipse
   .classpath
   .project
   .settings/

   # Application properties (exclude sensitive data)
   # application-prod.properties

   # OS
   .DS_Store
   Thumbs.db
   ```

3. Make initial commit:

   ```
   git add .
   git commit -m "Initial project setup - Lab 1"
   ```

4. Create GitHub/GitLab repository and push:

   ```
   git remote add origin <your-repository-url>
   git branch -M main
   git push -u origin main
   ```

Task 8: Create README.md

Description

Document your project with basic information.

Action Items

Create `README.md` in project root:

```
# [Your Project Name]

## Description
[Brief description of your project - 2-3 sentences]

## Technology Stack
- Java 17/21
- Spring Boot 3.2.x
- Maven
- H2 Database (development)
- PostgreSQL/MySQL (production)

## How to Run
1. Clone the repository
2. Open in IntelliJ IDEA
3. Run the main application class
4. Application starts on `http://localhost:8080`

## API Endpoints

### [Your Entity] Management
- `GET /api/tasks` - Get all items
- `GET /api/tasks/{id}` - Get single item
- `POST /api/tasks` - Create new item
- `PUT /api/tasks/{id}` - Update item
- `DELETE /api/tasks/{id}` - Delete item

## Development Status
- [x] Lab 1: Project setup and basic REST endpoints
- [ ] Lab 2: Database integration
- [ ] Lab 3: Service layer
- [ ] Lab 4: DTOs and validation
- [ ] Lab 5: Security
- [ ] Lab 6: Additional features
- [ ] Lab 7: Defense

## Author
[Your Name]
```

Checkpoint: What You Should Have

Before finishing Lab 1, verify you have:

Project topic chosen and approved
Spring Boot project created and running

Basic project structure (packages created)
At least one domain controller with CRUD endpoints
All endpoints tested in Postman
Git repository initialized
README.md created
Code committed to repository

Common Issues and Solutions

Issue: Application won't start

Solution: Check console for error messages. Common causes: - Port 8080 already in use → Change port in `application.properties` - Missing dependencies → Reload Maven project - Syntax errors in code → Fix compilation errors

Issue: 404 Not Found when testing endpoint

Solution: - Verify application is running - Check URL matches `@RequestMapping` and method mapping - Ensure controller has `@RestController` annotation - Check server port in application.properties

Issue: Lombok not working

Solution: - Enable annotation processing in IntelliJ: Settings → Build → Compiler → Annotation Processors → Enable - Install Lombok plugin for IntelliJ - Reload Maven project

Issue: H2 Console not accessible

Solution: - Verify `spring.h2.console.enabled=true` in properties - Access at: `http://localhost:8080/h2-console` - JDBC URL should match datasource URL in properties

---

Next Lab Preview

In Lab 2, you will: - Convert your models to JPA entities - Set up a real database (PostgreSQL/MySQL) - Create repositories - Implement actual database persistence - Test CRUD operations with real data

---

Optional Challenges

If you finish early, try these:

1. **Add query parameters:** Implement filtering (e.g., `/api/tasks?completed=true`)
2. **Add more entities:** Create controllers for 2-3 different entities
3. **Status codes:** Experiment with different HTTP status codes
4. **Exception handling:** Add basic try-catch blocks
5. **More HTTP methods:** Try PATCH for partial updates

Submission (Optional)

You may submit your progress on Microsoft Teams: - Link to your GitHub/GitLab repository - Brief description of what you implemented - Any questions or issues you encountered

No formal grading, but feedback will be provided to help you improve.

Resources

- Spring Boot Reference Documentation
- Spring Web MVC
- REST API Tutorial
- HTTP Status Codes

Good luck with your first lab!